

Structured Tool Representation Research (Multi-Turn Trajectory Learning)

This repository contains an undergraduate research prototype focused on **structured tool embeddings / tool representations** for smaller language models.

The core idea is not to build a production coding agent, but to study:

- how tool semantic normalization affects learning,
- how trajectory structuring affects behavioral signal quality,
- how compressed supervision affects tool-call consistency.

1) Research Context

Most tool-use data sources contain one or more issues:

- inconsistent tool names,
- noisy or repetitive outputs,
- weak continuity across multi-turn steps,
- high token cost with low supervision quality.

This project investigates whether **data-centric preprocessing** can improve learned structured tool-use behavior in smaller models.

Primary framing:

- representation quality over raw log scale,
- structured supervision over unfiltered trajectories,
- consistency of tool semantics over prompt-only formatting.

2) Research Objective

Evaluate whether normalized and compressed multi-turn tool trajectories can help a 3B model learn:

- consistent tool invocation patterns,
- cleaner tool semantics,
- stronger structured JSON trajectory outputs.

Coding-task outputs are used as **evaluation probes/artifacts**, not as the project's final product claim.

3) Repository Contents

Core implementation

- ``gen12.py`` - preprocessing, normalization, compression, filtering, scoring
- ``train.py`` - QLoRA supervised fine-tuning
- ``agentTester.py`` - structured JSON generation + parse validation

Analysis artifacts

- ``output1.json`` - valid structured output example
- ``outputF.json`` - invalid/truncated output example

Research notes

- `domain\_research.pdf` - domain framing and motivation
- `doc.txt` - detailed design rationale
- `milestone1.txt` - milestone summary
- `extra info.txt` - team contribution mapping

Supporting

- `requirements.txt`
- Empty placeholders: `agentTester.py`, `gen12.py`, `train.py`

4) Pipeline Summary

``text

Raw local SWE-chat style logs (./swe_chat_local)

- > preprocessing (normalize/compress/filter)
- > dataset_v8.jsonl
- > QLoRA fine-tuning (Qwen2.5-Coder-3B-Instruct)
- > adapter (./agent_lora)
- > structured JSON generation probe
- > valid/invalid output analysis

...

5) Implementation Highlights

A. Tool Semantic Normalization

Raw tool labels are mapped to a fixed schema:

- `read\_files`
- `search\_files`
- `modify\_files`
- `write\_files`
- `run\_command`

Why it matters:

- lowers tool entropy,
- improves semantic consistency in training signals.

B. Trajectory Structuring and Compression

The preprocessing stage performs:

- path normalization,
- low-signal output cleaning,
- repeated-step deduplication,
- command/output compression,
- session-level filtering/scoring.

Why it matters:

- preserves meaningful behavioral signal while reducing noise.

C. Structured Supervision

Final training data uses compact JSON trajectories (`goal`, `meta`, `m`, `end`) to provide consistent multi-turn supervision.

D. QLoRA Training

Base model: `Qwen/Qwen2.5-Coder-3B-Instruct`

Method: 4-bit + LoRA + SFT

6) Research Experience and Observations

From implementation and artifacts in this repo:

1. ****Preprocessing quality is central**.**

Data shaping had major practical impact on output structure quality.

2. ****Normalization improves consistency**.**

Canonical tool naming reduced representation fragmentation.

3. ****Compression is useful but lossy**.**

Shorter trajectories improve efficiency, but very detailed edit semantics can be reduced.

4. ****Structured outputs can still fail**.**

`output1.json` shows successful structure; `outputF.json` shows truncation/invalid behavior.

5. ****This is a research prototype**.**

Current evidence is mainly structural/qualitative in the included artifacts.

7) How to Run

Setup

```
```powershell
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```
```

Execution order

```
```powershell
python "gen12.py"
python "train.py"
python "agentTester.py"
```
```

Expected outputs:

- `dataset\_v8.jsonl`
- `./agent\_lora`
- `output.json` (if valid parse)

8) Scope and Boundaries

In scope

- structured tool trajectory preprocessing,
- supervised fine-tuning pipeline,
- output-structure probing.

Not claimed in this repository

- production autonomous system,
- deployment platform,
- complete benchmark suite with large-scale metrics.

9) Future Work

- stricter schema validation and constrained decoding,
- richer evaluation metrics for tool-call consistency,
- improved compressed edit representation,
- broader trajectory diversity and long-horizon scenarios.

10) Team Contribution Note

From `extra.info.txt`:

- Research and dataset pattern: Suhas, Swayam, Nikhil
- Dataset filtering/training/formatting collaboration: Shrikar, Dalip
- Tool-interaction pipeline: Harshit, Rahul

11) Quick Navigation

- Full technical details: `FULL\_PROJECT\_DOCUMENTATION.md`
- Report format: `PROJECT\_REPORT.md`
- Domain framing source: `domain\_research.pdf`